

```
In [ ]: # 4.2 Случайный лес (регрессия).
```

```
In [26]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
In [ ]: ## 1. Загрузка и обработка датасета.
Датасет, его загрузка и обработка как в заданиях 1.2, 2.2, 3,2.
```

```
In [27]: df_air = pd.read_csv(
    'C:/Users/Elizaveta/Downloads/air_quality/AirQuality.csv',
    delimiter=';',
    decimal=',',
    na_values=[' ', 'NA', 'nan', -200]
)
print(f"размер после загрузки: {df_air.shape}")
print("столбцы после загрузки:", list(df_air.columns))
df_air = df_air.loc[:, ~df_air.columns.str.contains('Unnamed')]
print(f"\nразмер таблицы после удаления пустых столбцов: {df_air.shape}")
print("столбцы:", list(df_air.columns))
print(df_air.head())
print(df_air.info())
print(df_air.isnull().sum())
```

размер после загрузки: (9471, 17)

столбцы после загрузки: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH', 'Unnamed: 15', 'Unnamed: 16']

размер таблицы после удаления пустых столбцов: (9471, 15)

столбцы: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH']

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	\
0	10/03/2004	18.00.00	2.6	1360.0	150.0	11.9	
1	10/03/2004	19.00.00	2.0	1292.0	112.0	9.4	
2	10/03/2004	20.00.00	2.2	1402.0	88.0	9.0	
3	10/03/2004	21.00.00	2.2	1376.0	80.0	9.2	
4	10/03/2004	22.00.00	1.6	1272.0	51.0	6.5	

	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	\
0	1046.0	166.0	1056.0	113.0	1692.0	1268.0	
1	955.0	103.0	1174.0	92.0	1559.0	972.0	
2	939.0	131.0	1140.0	114.0	1555.0	1074.0	
3	948.0	172.0	1092.0	122.0	1584.0	1203.0	
4	836.0	131.0	1205.0	116.0	1490.0	1110.0	

	T	RH	AH
0	13.6	48.9	0.7578
1	13.3	47.7	0.7255
2	11.9	54.0	0.7502
3	11.0	60.0	0.7867
4	11.2	59.6	0.7888

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 9471 entries, 0 to 9470

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	Date	9357 non-null	object
1	Time	9357 non-null	object
2	CO(GT)	7674 non-null	float64
3	PT08.S1(CO)	8991 non-null	float64
4	NMHC(GT)	914 non-null	float64
5	C6H6(GT)	8991 non-null	float64
6	PT08.S2(NMHC)	8991 non-null	float64
7	NOx(GT)	7718 non-null	float64
8	PT08.S3(NOx)	8991 non-null	float64
9	NO2(GT)	7715 non-null	float64
10	PT08.S4(NO2)	8991 non-null	float64
11	PT08.S5(O3)	8991 non-null	float64
12	T	8991 non-null	float64
13	RH	8991 non-null	float64
14	AH	8991 non-null	float64

dtypes: float64(13), object(2)

memory usage: 1.1+ MB

None

Date 114

Time 114

CO(GT) 1797

PT08.S1(CO) 480

NMHC(GT) 8557

C6H6(GT) 480

PT08.S2(NMHC) 480

NOx(GT) 1753

```
PT08.S3(NOx)      480
NO2(GT)           1756
PT08.S4(NO2)      480
PT08.S5(O3)       480
T                 480
RH                480
AH                480
dtype: int64
```

```
In [28]: X = df_air[feature_columns]
Y = df_air[target_column]
mask = X.notnull().all(axis=1)&Y.notnull()
X_clean = X[mask]
Y_clean = Y[mask]
print(f"Размер X: {X_clean.shape}")
print(f"Размер Y: {Y_clean.shape}")
print(f"\nЗначения целевой переменной {target_column}:")
print(f"Min: {Y_clean.min():.3f}")
print(f"Max: {Y_clean.max():.3f}")
print(f"Среднее: {Y_clean.mean():.3f}")
print(f"Медиана: {Y_clean.median():.3f}")
print(f"Стандартное отклонение: {Y_clean.std():.3f}")
```

```
Размер X: (6941, 11)
Размер Y: (6941,)
```

```
Значения целевой переменной C6H6(GT):
Min: 0.200
Max: 63.700
Среднее: 10.554
Медиана: 8.800
Стандартное отклонение: 7.465
```

```
In [29]: from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X_clean, Y_clean,
                                                    test_size=0.3,
                                                    random_state=42)

print(f"Обучающая выборка: {X_train.shape}")
print(f"Тестовая выборка: {X_test.shape}")
```

```
Обучающая выборка: (4858, 11)
Тестовая выборка: (2083, 11)
```

```
In [30]: from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X_clean, Y_clean, test_size=0.3,
                                                    random_state=42)
print(f"Обучающая выборка: {X_train.shape}")
print(f"Тестовая выборка: {X_test.shape}")
```

```
Обучающая выборка: (4858, 11)
Тестовая выборка: (2083, 11)
```

```
In [ ]: ## 2. Создание бейзлайна.
```

Ниже представлено построение базовой модели случайного леса RandomForestRegressor. Обучающая выборка (train) используется для обучения модели. Тестовая выборка (test) используется для оценки качества обученной модели. Используются метрики MAE, RMSE, R2. MAE=0.0216 показывает среднюю величину ошибки в тех же единицах, что и целевая переменная. RMSE=0.1470 также показывает среднюю ошибку переменной, но за счёт возведения в квадрат демонстрирует наличие выбросов. R2=0.9983 показывает долю дисперсии целевой переменной, полученное значение под базовой моделью демонстрирует высокие показатели, возможно переобучение. Она служ

```
In [11]: from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
RForest_Regressor = RandomForestRegressor(n_estimators=100)
RForest_Regressor.fit(X_train, Y_train)
Y_prediction = RForest_Regressor.predict(X_test)
MAE = mean_absolute_error(Y_test, Y_prediction)
RMSE = np.sqrt(mean_squared_error(Y_test, Y_prediction))
R2 = r2_score(Y_test, Y_prediction)
print(f"MAE: {MAE:.4f}")
print(f"RMSE: {RMSE:.4f}")
print(f"R2: {R2:.4f}")
```

MAE: 0.0216

RMSE: 0.1470

R2: 0.9983

```
In [ ]: ## 3. Улучшение бейзлайна.
Гипотеза 1: Подбор гиперпараметров.
С помощью RandomizedSearchCV подберём гиперпараметры для улучшения модели: колич
для разделения узла, минимальное число объектов в листе, количество признаков, р
из 50 на 5-кратной кросс-валидации. Полученные результаты:
Лучшие параметры: {'n_estimators': 300, 'min_samples_split': 2, 'min_samples_lea
Лучший отрицательный MSE на CV: -0.015998078888576905
Тестовые метрики: MAE = 0.0216, RMSE = 0.3056, R2 = 0.9984
Полученные значения метрик говорят о том, что базовая модель показывает хорошие
```

```
In [20]: from sklearn.model_selection import RandomizedSearchCV, KFold
cv = KFold(n_splits=5, shuffle=True, random_state=42)
param_dist_reg = {
    'n_estimators': [100, 200, 300, 500],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', 0.3, 0.5, 1.0],
    'criterion': ['squared_error', 'absolute_error', 'friedman_mse']
}
rf_reg = RandomForestRegressor(random_state=42, n_jobs=-1)
random_search_reg = RandomizedSearchCV(
    rf_reg,
    param_distributions=param_dist_reg,
    n_iter=50,
    cv=cv,
    scoring='neg_mean_squared_error',
    random_state=42,
    n_jobs=-1,
    verbose=1
)
random_search_reg.fit(X_train, Y_train)
print("Лучшие параметры:", random_search_reg.best_params_)
print("Лучший отрицательный MSE на CV:", random_search_reg.best_score_)
best_rf_reg = random_search_reg.best_estimator_
best_rf_reg.fit(X_train, Y_train)
y_pred_reg = best_rf_reg.predict(X_test)
mae = mean_absolute_error(Y_test, y_pred_reg)
rmse = np.sqrt(mean_squared_error(Y_test, y_pred_reg))
r2 = r2_score(Y_test, y_pred_reg)
print(f"Тестовые метрики: MAE = {mae:.4f}, RMSE = {rmse:.4f}, R2 = {r2:.4f}")
```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

Лучшие параметры: {'n_estimators': 300, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_features': 1.0, 'max_depth': 10, 'criterion': 'friedman_mse'}

Лучший отрицательный MSE на CV: -0.015998078888576905

Тестовые метрики: MAE = 0.0216, RMSE = 0.3056, R2 = 0.9984

In []: *## 4. Имплементация алгоритма машинного обучения.*

Был создан собственный алгоритм случайного леса на основе ранее оаработанного а
Это ансамблевый метод, строящий множество решающих деревьев на основе различных
признаков для каждого разбиения, что снижает корреляцию между деревьями. Основны
__init__ - задание гиперпараметров.
fit - строит лес из n_estimators деревьев.
_impurity - вычисляет меру неопределённости для массива меток.
predict - формирование предсказаний.

```
In [21]: import numpy as np
class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None, prob
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value
class MyDecisionTreeRegressor:
    def __init__(self, max_depth=None, min_samples_split=2, min_samples_leaf=1,
        criterion='squared_error', max_features=None, random_state=None
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion.lower()
        self.max_features = max_features
        self.random_state = random_state
        self.root = None
        self.n_features_ = None
        self.rng = np.random.RandomState(random_state)
    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y)
        self.n_features_ = X.shape[1]
        self.root = self._build_tree(X, y, depth=0)
        return self
    def _build_tree(self, X, y, depth):
        n_samples = X.shape[0]
        var_y = np.var(y)
        if (depth == self.max_depth) or (n_samples < self.min_samples_split) or
            leaf_value = np.mean(y)
            return Node(value=leaf_value)
        best_feat, best_thresh, best_gain = self._best_split(X, y)
        if best_feat is None:
            leaf_value = np.mean(y)
            return Node(value=leaf_value)
        left_mask = X[:, best_feat] <= best_thresh
        right_mask = ~left_mask
        left_child = self._build_tree(X[left_mask], y[left_mask], depth + 1)
        right_child = self._build_tree(X[right_mask], y[right_mask], depth + 1)
        return Node(feature=best_feat, threshold=best_thresh, left=left_child, r
    def _best_split(self, X, y):
        best_gain = -float('inf')
        best_feature = None
        best_threshold = None
```

```

current_var = np.var(y)
n_features = X.shape[1]
if self.max_features is None or self.max_features >= n_features:
    features_to_try = range(n_features)
else:
    features_to_try = self.rng.choice(n_features, size=self.max_features)
for feature in features_to_try:
    thresholds = np.unique(X[:, feature])
    for thresh in thresholds:
        left_mask = X[:, feature] <= thresh
        right_mask = ~left_mask
        if np.sum(left_mask) < self.min_samples_leaf or np.sum(right_mas
            continue
        left_var = np.var(y[left_mask])
        right_var = np.var(y[right_mask])
        n_left = np.sum(left_mask)
        n_right = np.sum(right_mask)
        n_total = n_left + n_right
        gain = current_var - (n_left / n_total) * left_var - (n_right /
        if gain > best_gain:
            best_gain = gain
            best_feature = feature
            best_threshold = thresh
if best_gain <= 0:
    return None, None, None
return best_feature, best_threshold, best_gain
def predict(self, X):
    X = np.array(X)
    predictions = []
    for x in X:
        node = self.root
        while node.value is None:
            if x[node.feature] <= node.threshold:
                node = node.left
            else:
                node = node.right
        predictions.append(node.value)
    return np.array(predictions)

```

```

In [22]: class MyRandomForestRegressor:
    def __init__(self, n_estimators=100, max_depth=None, min_samples_split=2,
        min_samples_leaf=1, criterion='squared_error', max_features='sq
        bootstrap=True, random_state=None):
        self.n_estimators = n_estimators
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.criterion = criterion
        self.max_features = max_features
        self.bootstrap = bootstrap
        self.random_state = random_state
        self.trees = []
    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y)
        n_samples, n_features = X.shape
        if isinstance(self.max_features, str):
            if self.max_features == 'sqrt':
                max_features_val = int(np.sqrt(n_features))
            elif self.max_features == 'log2':

```

```

        max_features_val = int(np.log2(n_features))
    elif self.max_features == 'auto':
        max_features_val = n_features
    else:
        raise ValueError("max_features должен быть 'sqrt', 'log2', 'auto'")
    elif isinstance(self.max_features, float):
        max_features_val = int(self.max_features * n_features)
    else:
        max_features_val = self.max_features
    if max_features_val is None or max_features_val > n_features:
        max_features_val = n_features
    if max_features_val < 1:
        max_features_val = 1
    rng = np.random.RandomState(self.random_state)
    for i in range(self.n_estimators):
        if self.bootstrap:
            indices = rng.choice(n_samples, size=n_samples, replace=True)
        else:
            indices = np.arange(n_samples)
        X_boot = X[indices]
        y_boot = y[indices]
        tree = MyDecisionTreeRegressor(
            max_depth=self.max_depth,
            min_samples_split=self.min_samples_split,
            min_samples_leaf=self.min_samples_leaf,
            criterion=self.criterion,
            max_features=max_features_val,
            random_state=rng.randint(0, 10**6)
        )
        tree.fit(X_boot, y_boot)
        self.trees.append(tree)
    return self

def predict(self, X):
    X = np.array(X)
    preds = []
    for tree in self.trees:
        pred = tree.predict(X)
        preds.append(pred)
    return np.mean(preds, axis=0)

```

In []: Тестирование собственного алгоритма. Результаты:
 MAE = 0.0260, RMSE = 0.3056, R2 = 0.9984
 Полученные результаты идентичны значениям метрик на библиотечной модели, что гов

```

In [24]: my_rf_reg = MyRandomForestRegressor(
    n_estimators=300,
    max_depth=10,
    min_samples_split=2,
    min_samples_leaf=1,
    criterion='squared_error',
    max_features='auto',
    bootstrap=True,
    random_state=42
)
my_rf_reg.fit(X_train, Y_train)
y_pred_reg = my_rf_reg.predict(X_test)
mae = mean_absolute_error(Y_test, y_pred_reg)
rmse = np.sqrt(mean_squared_error(Y_test, y_pred_reg))
r2 = r2_score(Y_test, y_pred_reg)

```

```
print("MyRandomForest:")  
print(f"MAE = {mae:.4f}, RMSE = {rmse:.4f}, R2 = {r2:.4f}")
```

MyRandomForest:

MAE = 0.0260, RMSE = 0.3056, R2 = 0.9984

In []: *## Выводы:*
В ходе исследования алгоритма случайного леса для регрессии была обучена базовая метрик MAE: 0.0216, RMSE: 0.1470, R2: 0.9983, свидетельствующие о высоком качестве. Изменились следующим образом: MAE = 0.0216, RMSE = 0.3056, R2 = 0.9984. Далее было построено дерево из предыдущего задания. Результаты тестирования собственного алгоритма MAE реализации (полное совпадение с результатами на библиотечной модели).